SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES

(Declared as Deemed to be University under Section 3 of UGC Act 1956)

CO2_AT3 Error Analysis Task

COURSE NAME: Software Testing

COURSE CODE: CSA37

COURSE OUTCOME COVERED

CO2: Design effective test cases using black-box, white-box testing techniques and to control methods flow.

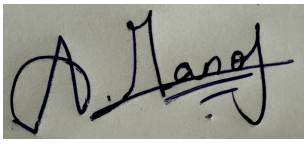
Assessment Tool Weightage: 20%

Code of Conduct:

I Manoj A / 192421295 that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Question and Answers

1. A tester designs test cases for an online shopping system using equivalence partitioning but groups valid and invalid inputs together and ignores boundary values. Identify all errors in the partitioning and BVA, examine the root causes, propose corrected test cases, and justify the improvement in defect detection.

Answer:

Consider the quantity field of the shopping cart, where the requirement says that a customer can order 1 to 100 units of an item. The tester prepared two classes: Class A (−5 to 50) and Class B (51 to 150), with one test value in each (for example 20 and 120). The design is analysed below.

i) Errors identified in the test design

- **Mixed classes:** Class A combines invalid values (−5 to 0) with valid values (1 to 50), and Class B combines valid values (51 to 100) with invalid values (101 to 150). An equivalence class must contain only values that the program treats in the same way.
- **Wrong representative values:** the test value 20 represents the whole of Class A, so the invalid values −5 to 0 are never tested; the value 120 represents Class B, so the valid values 51 to 100 are never tested.
- **Missing invalid classes:** non-numeric input, blank input, decimal quantities (2.5) and very large numbers have no class.
- **No boundary values:** 0, 1, 100 and 101 are not tested, although defects occur most often at these points.
- **No expected results:** the design does not state what the system should display for each class (error message or acceptance).

ii) Root causes

- The tester divided the numbers into convenient ranges (halves) instead of deriving the classes from the requirement of 1 to 100.
- The principle that a class has one expected behaviour was not understood, so valid and invalid values were mixed.
- Boundary Value Analysis was treated as optional and was not applied after the classes were defined.
- Only valid numeric input was considered, so negative testing was not planned.
- The design was not reviewed against the requirement or by a second person.

iii) Corrected test cases

- EC1 (invalid): quantity ≤ 0, representative -3 → error “Quantity must be at least 1”.
- EC2 (valid): quantity 1 to 100, representative 25 → item added to the cart.
- EC3 (invalid): quantity > 100, representative 150 → error “Maximum 100 per order”.

- EC4 (invalid): non-numeric, blank or decimal, representatives “abc”, blank, 2.5 → input rejected with a message.
- Boundary tests: 0 (rejected), 1 (accepted), 2 (accepted), 99 (accepted), 100 (accepted), 101 (rejected).
- Combination check: quantity 100 with stock of only 60 units → message “Only 60 available”.

iv) Justification

The corrected classes are disjoint and cover the whole input domain, so each requirement rule is checked and no input type is forgotten. The boundary tests target the places where relational operators are most often miscoded (for example > 100 written instead of >= 100). Eleven focused tests now replace two ineffective tests, which means that defects at the limits and in invalid input handling are detected, test coverage can be explained to others, and the risk of wrong orders in production is reduced.

2. A QA engineer performs random testing on a banking application and claims high coverage without any structured technique. Critical defects in transaction limits and boundaries are missed. Evaluate the approach, identify errors in strategy and coverage assumptions, analyse root causes, and suggest black-box techniques with justification.

Answer:

Random testing means choosing inputs without a design, and the QA engineer measured coverage by the number of tests executed. This gives no information about which requirements, input classes or boundaries were checked.

i) Errors in the strategy and in the coverage assumptions

- **Number of tests treated as coverage:** a thousand random transfers of ₹100 to ₹5,000 test only one class many times, and the coverage of limits is zero.
- **No link to requirements:** nobody can show that the daily limit, the per-transaction limit, the minimum balance or the OTP rule were verified.
- **Boundary values missed:** the probability that a random amount equals exactly ₹50,000 or ₹50,001 is very small.
- **Not repeatable:** the same random tests cannot be re-run after a fix, so regression is not possible.
- **No negative and combination tests:** invalid amounts, insufficient balance, duplicate submission and limit reached after several transactions were not planned.
- **No expected results:** without a prepared oracle the tester cannot recognise a wrong result.

ii) Root causes of the missed defects

- Lack of knowledge of the structured techniques, or the pressure of time, led to unplanned testing.
- Test adequacy was not defined, so “high coverage” was a personal impression.
- Test design was skipped because requirements were not analysed for limits and rules.
- Defects cluster at boundaries and in rare combinations, which random selection rarely reaches.

iii) Appropriate black-box techniques

- **Equivalence Partitioning:** classes for amount (below minimum, valid, above maximum, invalid format), account state (active, frozen, closed) and balance (sufficient, equal, insufficient).
- **Boundary Value Analysis:** amounts at minimum − 1, minimum, minimum + 1, maximum − 1, maximum, maximum + 1 (for example ₹0, ₹1, ₹2, ₹49,999, ₹50,000, ₹50,001), and the same for daily limit and OTP expiry.
- **Decision Table Testing:** combinations of amount within limit, balance sufficient, payee verified and OTP valid.
- **State Transition Testing:** the transaction life cycle (initiated, authorised, completed, failed, reversed) and the account states.
- **Risk-based prioritisation:** money-moving functions are tested first and most deeply.

iv) Justification

Each technique produces a documented list of test cases that can be traced to a requirement, so test adequacy is measurable, for example 100% of equivalence classes and 100% of boundaries covered. Boundary and decision-table tests find precisely the limit and combination defects that were missed. Because the tests are fixed and repeatable, they become a regression suite and the reliability of the banking application can be demonstrated. Random testing may still be used in a small exploratory session after the structured tests are complete.

3. A developer performs white-box testing but creates an incomplete control flow graph, missing decision nodes and loops, resulting in poor path coverage. Analyse the errors, identify root causes, propose a proper control flow representation with all independent paths, and justify the improvement.

Answer:

The following module is used to explain the errors.

- 1: read n; sum = 0; i = 1
- 2: while (i ≤ n) (D1)

- 3: if (a[i] < 0) negatives = negatives + 1 (D2); else if (a[i] = 0) zeros = zeros + 1 (D3); else sum = sum + a[i]
- 4: i = i + 1
- 5: if (sum > 100) display “High” else display “Normal” (D4)

i) Errors in the developer's control flow graph

- The back edge from node 4 to the loop test (D1) is missing, so the graph is drawn as a straight sequence and the loop is not visible.
- The loop exit edge (when i is greater than n) is missing, so the case n = 0, where the loop body is never executed, does not exist in the graph.
- The else-if (D3) is shown as a single block with D2, so the zero case is not a separate node.
- The final decision D4 is missing, so only one of the two displayed messages is represented.
- Nodes are drawn per line of text and not per basic block, and decision nodes do not have separate true and false edges.

ii) Root causes for the inadequate coverage

- The graph was drawn from the logic as the developer remembered it, and not from the code, so loops and nested conditions were overlooked.
- Cyclomatic complexity was not calculated, so the expected number of independent paths (the target) was unknown.
- Because the graph was incomplete, the tests, which were designed from it, could only reach a part of the paths.
- Compound structures (while with if–else if–else inside) were not decomposed into elementary decisions.

iii) Corrected control flow representation and independent paths

The corrected graph has four decision nodes (D1, D2, D3, D4), so V(G) = 4 + 1 = 5. The loop-back edge goes from node 4 to D1, D1 false goes to D4, and each decision has both true and false edges. The independent paths are:

- P1: n = 0 → D1 false → D4 false → display “Normal”.
- P2: n ≥ 1, a[1] negative → D2 true → increment → D1 false → D4 false.
- P3: a[1] = 0 → D2 false, D3 true → increment → D1 false → D4 false.
- P4: a[1] positive with sum ≤ 100 → D2 false, D3 false → add to sum → D1 false → D4 false.
- P5: a[1] positive with sum > 100 (for example 150) → D4 true → display “High”.
- Additional loop tests: n = 1, 2, typical and maximum, and one array that contains negative, zero and positive values in the same run (the loop executed several times with different decisions).

iv) Justification

A correct graph exposes all decisions, loops and exits, gives the target number of paths, and allows test cases to be designed for each path. The five tests above reach 100% statement and branch coverage, which the incomplete graph could not give. Defects in the zero-iteration case, in the handling of zero values, and in the final threshold comparison are therefore detected, and the coverage can be reported with a number that others can check.

4. A testing team reports 100% statement coverage but fails to detect logical errors in conditional statements. Evaluate the adequacy of the testing, identify the gaps in the coverage criteria, explain why statement coverage alone was insufficient, and recommend branch or path coverage.

Answer:

Statement coverage shows only that every line was run. It does not show that every outcome of a condition has been used, and it does not prove that the results are correct. The example used here is a discount rule: if (amount ≥ 1000 OR member) discount = 10%.

i) Evaluation of the testing performed and gaps

A single test, amount ₹1,500 and a non-member, makes the condition true and executes the discount statement, so statement coverage is 100%. The tests are inadequate because of the following gaps.

- The false outcome of the decision was never executed, so the case with no discount is untested.
- The boundary value of exactly ₹1,000 was not tested, so an error where the code uses > instead of ≥ is not detected (the test value ₹1,500 passes either way).
- The condition member was never shown to affect the result; if the code ignores membership, the test still passes.
- There was no check of the calculated discount, only of the execution of the statement.

ii) Why statement coverage alone was insufficient

- A decision without an else part has no statement for the false branch, so the branch can be skipped completely while coverage stays at 100%.
- In a compound condition, only the combination used is evaluated, and the other conditions are not tested.
- Statement coverage does not require tests at relational boundaries, where most logical errors are made.
- It is not sensitive to wrong operators, wrong constants or wrong order of conditions as long as the lines are run.
- Missing code (for example a missing else or a missing rule) cannot be found by a measure that considers only the code that exists.

iii) Recommended white-box strategy

- **Branch (decision) coverage:** test the condition true (₹1,500) and false (₹500, non-member); this gives at least two tests.
- **Condition coverage and MC/DC:** (amount 1,500, non-member) true, (amount 500, member) true, (amount 500, non-member) false; the first and third differ only in the amount, the second and third only in membership, so each condition is shown to affect the result.
- **Boundary tests:** ₹999, ₹1,000, ₹1,001 with a non-member.
- **Path coverage for sequences of decisions:** when the module has several decisions (discount, tax, shipping), test the combinations of their outcomes, at least all basis paths.
- **Check results:** compute the expected discount independently, and use a coverage tool to report statement, branch and condition coverage, with branch coverage required to be 100% before release.

iv) Justification

Branch and condition coverage force the tester to execute every decision outcome and show the effect of every condition, so wrong operators, ignored conditions and missing else cases are exposed. Path coverage adds interaction faults between decisions. The revised approach therefore detects the logical errors that statement coverage hid, and gives a stronger and more meaningful measure of test adequacy.

5. A tester uses Equivalence Class Partitioning for an age field (0–100) but defines only two classes, negative and positive numbers. Analyse the errors, explain why valid and invalid limits and BVA are missing, and propose corrected partitions with justification.

Answer:

i) Errors in the partitioning strategy

- **Zero is not covered:** zero is neither negative nor positive, and it is a valid age, so an important value belongs to no class.
- **The positive class mixes valid and invalid values:** positive numbers 1 to 100 are valid, but 101, 150 and 1000 are positive and invalid; one representative value (for example 30) cannot represent both.
- **The upper limit is not represented:** ages above 100 are not rejected in the tests, so a missing upper-limit check is not detected.
- **Other invalid inputs are missing:** decimals (25.5), non-numeric text, blank input, special characters and very large numbers.
- **No boundary values:** −1, 0, 1, 99, 100 and 101 are not tested.

ii) Why this partitioning fails

An equivalence class must be based on the requirement, which states that the valid age is from 0 to 100. The sign of a number is not a rule of the requirement, so the classes do not divide the inputs into groups that the program treats differently. As a result, the valid class is not separated from the invalid class on the upper side, and the program could accept an age of 500 without any test failing. Because there is no Boundary Value Analysis, off-by-one defects such as age > 0 instead of age >= 0, or age < 100 instead of age <= 100, would remain.

iii) Corrected partitions

- EC1 (invalid): age < 0, representative −5 → rejected with an error.
- EC2 (valid): age from 0 to 100, representative 45 → accepted.
- EC3 (invalid): age > 100, representative 150 → rejected with an error.
- EC4 (invalid): decimal numbers, representative 25.5 → rejected (if only integers are allowed).
- EC5 (invalid): non-numeric, special characters or blank, representatives “abc”, “#”, blank → rejected with a clear message.
- EC6 (invalid): extremely large numbers, representative 99999999999 → rejected without overflow or crash.

iv) Boundary value tests

- −1 (invalid, just below the minimum), 0 (valid, minimum), 1 (valid, just above the minimum).
- 99 (valid, just below the maximum), 100 (valid, maximum), 101 (invalid, just above the maximum).

v) Justification

The corrected partitions are separate, cover every possible input and each has one expected result, so the validation rule of 0 to 100 is checked completely. The boundary tests find off-by-one errors, the invalid classes show that incorrect data is rejected safely, and the whole set of twelve tests is small enough to run in every regression cycle. Input validation is therefore more robust, and bad data cannot enter the system through the age field.

6. A developer claims full Condition Coverage for a complex if(A && B …) statement. (The question text is cut off in the supplied file; it is answered on its evident meaning: full condition coverage claimed, but decision errors remain.) Analyse the error in the claim, identify root causes, and propose corrected tests.

Answer:

Condition coverage requires that each atomic condition (A and B) takes the value true and the value false at least once. It does not require that the whole decision takes both outcomes. This is the likely error in the developer's claim. The statement analysed is: if (A && B) { action }.

i) Errors in the developer's claim

- The developer used two tests: (A = true, B = false) and (A = false, B = true). Each condition has been true once and false once, so “condition coverage” is 100%.
- In both tests the whole decision (A && B) is false, so the action inside the if is never executed. Decision (branch) coverage is only 50% and statement coverage of the action is 0%.
- The case where both conditions are true, the only case that executes the action, was never tested.
- Individual conditions have not been shown to affect the result independently, so a wrong operator (|| instead of &&) or a wrongly coded condition could remain.

ii) Root causes

- The definition of condition coverage was misunderstood as covering the whole decision.
- The tests were chosen to give each condition both values with the smallest number of tests, without checking the outcome of the decision.
- No coverage tool report for decision or MC/DC coverage was used, so the gap was not visible.
- Compound decisions were not analysed with a truth table.

iii) Corrected tests

The truth table of A && B has four rows: (T,T) → true; (T,F) → false; (F,T) → false; (F,F) → false. The corrected set is:

- Test 1: A = true, B = true → decision true, action executed.
- Test 2: A = false, B = true → decision false (A alone changed the result compared with Test 1).
- Test 3: A = true, B = false → decision false (B alone changed the result compared with Test 1).

These three tests achieve condition coverage, decision (branch) coverage and MC/DC. For a larger expression such as (A && B) || C, n + 1 tests are used (for four conditions, five tests) so that each condition is shown to independently change the outcome, instead of testing all 2^n combinations.

iv) Justification

The corrected tests execute the action, test both outcomes of the decision, and prove that each condition influences the decision. Faulty operators, ignored conditions and wrong conditions are therefore detected, which the developer's two tests could not show. This is why standards for safety-critical software ask for MC/DC and not condition coverage alone.

7. A QA team applies Decision Table Testing for a discount system but misses combinations of "Premium Member" and "Coupon Applied", leading to price errors. Analyse the errors in the decision table, identify why it is incomplete, and propose a revised exhaustive decision table with justification.

Answer:

Assume the business rules: a Premium Member receives 10% discount; a valid coupon gives 5% extra discount; when both apply the discounts are added (15%); an expired or invalid coupon is rejected with a message and gives no discount. The price used in the examples is ₹1,000.

i) Errors in the construction of the original table

- Only the two extreme rules were written: member with coupon, and non-member without coupon. The mixed cases (member without coupon, non-member with coupon) were never listed.
- Coupon validity (valid or expired) was not treated as a condition, so the invalid-coupon behaviour is not tested.
- The number of rules was not checked against 2^n, so the missing rules were not noticed.
- Expected actions were not stated for all rules, particularly the error message and the stacking of the discounts.

ii) Why the approach was incomplete

A decision table is complete only if every combination of the condition values appears in a rule, or is explicitly combined by a 'don't care' entry. The team started from the typical cases and did not enumerate. The missing combinations are exactly those in which the code could apply the wrong discount, for example giving the coupon discount to a non-member only when the premium flag is also set, or replacing instead of adding the discounts.

iii) Revised exhaustive decision table

Conditions: C1 = Premium Member (Y/N), C2 = Coupon applied (Y/N), C3 = Coupon valid (Y/N, relevant only when C2 = Y). The full table has $2 \times 2 \times 2 = 8$ combinations, of which the two combinations with C2 = N and C3 = Y or N give the same result, so six distinct rules remain:

- R1: C1 = N, C2 = N → no discount, total ₹1,000.
- R2: C1 = Y, C2 = N → 10% discount, total ₹900.
- R3: C1 = N, C2 = Y, C3 = Y → 5% coupon discount, total ₹950.
- R4: C1 = N, C2 = Y, C3 = N → coupon rejected with message, total ₹1,000.
- R5: C1 = Y, C2 = Y, C3 = Y → 10% + 5% = 15%, total ₹850.
- R6: C1 = Y, C2 = Y, C3 = N → member discount only, coupon rejected with message, total ₹900.

Additional tests are added for boundary cases: coupon expiring on the day of purchase and on the day before and after; the same coupon applied twice; an order of ₹0; a discount that would make the total negative; and rounding of the discounted amount to two decimals.

iv) Justification

All eight raw combinations are accounted for, each rule has a test and an independently calculated expected total, and the invalid coupon cases are included. The table can be reviewed against the requirement by the business, which prevents misunderstanding. Because every combination of membership and coupon is tested, price calculation errors are detected before release, and the table is kept as a living document for the regression suite when new discount rules are added.

8. During unit testing of a loop, a tester checks only the first and last iterations (n = 1 and n = max). Defects at n = 0 and n = max + 1 are missed. Evaluate the errors in the loop testing strategy, analyse root causes, recommend a corrected approach for simple and nested loops, and justify how it ensures path integrity.

Answer:

i) Errors in the loop testing strategy

- **Zero iterations not tested:** n = 0 skips the loop body completely; defects such as use of uninitialised results, division by zero (average of zero items) or an empty-input crash are missed.
- **Overrun not tested:** n = max + 1 checks whether the loop stops or whether the program writes outside the array or accepts an invalid count; this was not tested.
- **Middle values ignored:** n = 2 (the first case in which the second iteration depends on the first) and a typical n are not tested, so defects that depend on the state left from one iteration are missed.
- **Only the count was treated as input:** the contents of the iterations (valid and invalid data) and exits from inside the loop (break, return, exception) were not varied.

ii) Root causes

- The tester assumed that if the first and last iterations work, all the intermediate ones work as well.
- Zero and max + 1 are outside the “normal” range, so they were not considered as test inputs.
- No standard checklist for loop testing was used, and the control flow graph with the loop-back edge and the exit edge was not drawn.
- Nested loops were not considered; the cost of testing every combination was feared, so only extremes were tried.

iii) Corrected approach for simple loops

- Skip the loop entirely (n = 0).
- Exactly one pass (n = 1).
- Two passes (n = 2).
- A typical number of passes (n = m, where m is less than max).
- n = max − 1, n = max and n = max + 1 passes (the last must be rejected or handled safely).
- Exit tests: leave the loop early with break or return, and an exception inside the loop, to verify the clean-up.

iv) Corrected approach for nested loops

- Start with the innermost loop, while all outer loops are at their minimum values; apply the simple loop tests to it.
- Then test the next loop outward, keeping the inner loop at typical values and the further outer loops at minimum values.
- Continue outward until all loops have been tested, and finally run a test with all loops at typical values and one with all at maximum values.
- For concatenated loops (one after the other) that share variables, test them as nested loops; if they are independent, test them as simple loops.

v) Justification

These tests execute the loop-entry decision both ways, the loop-back edge once and many times, and the exit edge at the minimum, normal and maximum counts, so the paths through the loop are tested without trying every combination. In a nested structure, the method keeps the number of tests manageable (it grows by the sum, not the product, of the loop counts). Defects at n = 0 and at max + 1, which the tester missed, are detected, and the integrity of the paths through the loop is verified.

9. An engineer uses Error Guessing as the primary technique for a safety-critical braking system, ignoring structured black-box methods. Identify the errors in relying on ad-hoc techniques, analyse the impact on adequacy, and suggest a combination of BVA and State Transition Testing with justification.

Answer:

i) Errors in relying only on error guessing

- Error guessing depends on the experience of the tester; it cannot be shown to be complete, and two testers will produce different tests.
- There is no traceability to the safety requirements, so it cannot be proved that every hazard (for example failure to brake, brake locking or delayed braking) was tested.
- There is no measure of adequacy, and no criterion tells when the testing can stop.

- Boundary and sequence faults (pedal pressure at the threshold, brake pressed during a fault) may not be thought of.
- The tests are neither documented nor repeatable, which is not acceptable for a safety case and for audits under functional safety standards such as ISO 26262.

ii) Impact on test adequacy

Large parts of the input space (speed, pedal pressure, wheel-slip ratio, temperature, sensor values) and of the system behaviour (normal braking, ABS action, failure and fallback) remain unverified. The testing may appear busy while whole classes of situations are untested, and a rare combination, such as a sensor fault during hard braking at high speed, can then be first discovered on the road. For a safety-critical system this means unacceptable residual risk and no evidence for certification.

iii) Corrected strategy: Boundary Value Analysis

- Pedal pressure: 0%, 1%, 2%, 49%, 50%, 51% (ABS threshold), 99%, 100% and 101% (sensor over-range).
- Vehicle speed: 0, 1, 5 (low-speed limit of ABS), 119, 120, 121 km/h, and the maximum rated speed ± 1.
- Wheel slip ratio at the ABS activation threshold: threshold − 1, threshold, threshold + 1.
- Sensor values: minimum and maximum of the valid range, one beyond each limit, a missing signal and a stuck value.
- Response time: requirement (for example 100 ms), and values just inside and outside the limit.

iv) Corrected strategy: State Transition Testing

The states are Released, Applying, Holding, ABS Active, Fault Detected and Safe Fallback (mechanical or reduced braking). The transitions that are tested are: Released → Applying (pedal pressed), Applying → Holding (pedal held), Applying → ABS Active (slip above the threshold), ABS Active → Applying (slip returns to normal), Holding or Applying → Released (pedal released), any state → Fault Detected (sensor or actuator failure), and Fault Detected → Safe Fallback.

- Test every valid transition at least once (0-switch coverage) and common sequences of two transitions (for example Applying → ABS Active → Applying).
- Test the illegal transitions that must be rejected, such as Released → ABS Active without pedal input, or Safe Fallback → ABS Active without a reset.
- Test the interruption cases: fault during ABS action, release during ABS, pedal pressed again while the previous cycle has not finished, power dip.
- Keep error guessing only as a supplement, for example from previous accident reports.

v) Justification of increased safety assurance

BVA verifies exactly the thresholds at which the control logic changes its behaviour, and State Transition Testing verifies the correctness of the behaviour in sequence, including faults and recovery, which is where the most dangerous hazards lie. The tests derive from the safety requirements, are repeatable and measurable (all states, all transitions and all boundaries covered), and form evidence for the safety case. The structured approach therefore gives a much higher and demonstrable level of confidence than error guessing alone.

10. A tester designs a CFG but fails to represent the else branches of multiple nested if statements, giving 100% statement coverage but 0% branch coverage on error paths. Analyse the errors in the graph construction, identify the root causes, and propose a corrected CFG and independent paths to ensure branch adequacy.

Answer:

The following withdrawal logic is used as an example. Note that the if statements have no explicit else part.

- 1: if (user is authenticated) (D1) {
- 2: if (amount > 0) (D2) {
- 3: if (balance ≥ amount) (D3) {
- 4: balance = balance − amount; display “Success” } } }
- 5: return

i) Errors in the graph construction

- The tester drew only the true edges: 1 → 2 → 3 → 4 → 5, a single straight line with no false edges.
- The three implicit else branches (not authenticated, amount not positive, insufficient balance) were not drawn, so the paths through them do not exist in the graph.
- Join (merge) nodes were not shown, so the graph does not show that all false edges go directly to node 5.
- The decision nodes were treated as ordinary statements, with only one outgoing edge.

ii) Why statement coverage is 100% and error-path branch coverage is 0%

A single successful withdrawal executes nodes 1 to 5, so every statement has been run, and statement coverage is 100%. The implicit else branches contain no statement, so nothing is left unexecuted from the point of view of statements. The decisions have six outcomes (three true and three false); the test covered only the three true outcomes, which is 50% branch coverage in total, and the false outcomes, which are the error paths, were covered 0%.

iii) Root causes for the missing nodes

- The graph was drawn from the statements and the indentation, and not from the decisions; when there is no else statement, nothing was drawn.
- The concept that an if without else still has a false outcome that leaves the decision was not applied.
- The same graph was used to design the tests, so the missing branches were never tested, and the tool results (statement coverage) hid the omission.
- The code itself is defective: the missing else parts mean that errors are ignored silently, and the user receives no message; the poor design made the graph look simple.

iv) Corrected CFG and independent paths

The corrected graph has three decision nodes with two outgoing edges each, and V(G) = 3 + 1 = 4. False edges from D1, D2 and D3 all go to node 5 (the return), either directly or, after correction of the code, through new error nodes (E1, E2, E3) that display an error message.

- P1: D1 false → (E1: “Authentication failed”) → 5. Test: unauthenticated user.
- P2: D1 true → D2 false → (E2: “Invalid amount”) → 5. Test: authenticated user, amount 0 or −100.
- P3: D1 true → D2 true → D3 false → (E3: “Insufficient balance”) → 5. Test: authenticated user, amount ₹5,000, balance ₹1,000.
- P4: D1 true → D2 true → D3 true → 4 → 5. Test: authenticated user, amount ₹500, balance ₹1,000 → success, balance ₹500.
- Boundary test for D3: amount equal to balance (accepted) and amount equal to balance + 1 (rejected).

v) Justification

The four paths give 100% statement coverage and 100% branch coverage, including all error paths, and the corrected code returns a message for each failure, so that the tests can check the result. The same method (explicit false edges and merge nodes for every decision, with V(G) as the target) prevents the omission in the future, and defects in the handling of invalid users, invalid amounts and insufficient balance are detected before release.

RUBRICS FOR CALCULATION:

Error Identification (20) – Level 4 (Exemplary): Accurately identifies all errors with clear understanding of issues; Level 3 (Proficient): Identifies most errors with minor omissions; Level 2 (Developing): Identifies limited errors; some are missed; Level 1 (Beginning): Fails to identify key errors.

Root Cause Analysis (30) – Level 4 (Exemplary): Clearly explains underlying causes with strong technical reasoning; Level 3 (Proficient): Explains causes with moderate clarity; Level 2 (Developing): Partial explanation; weak reasoning; Level 1 (Beginning): Unable to identify causes of errors.

Correction & Justification (25) – Level 4 (Exemplary): Provides correct solutions with strong justification and logic; Level 3 (Proficient): Provides correct corrections with some justification; Level 2 (Developing): Corrections are partially correct or weakly justified; Level 1 (Beginning): Incorrect or no corrections provided.

Application of Concepts (15) – Level 4 (Exemplary): Effectively applies relevant concepts to analyze and resolve errors; Level 3 (Proficient): Applies concepts with minor errors; Level 2 (Developing): Limited application; requires guidance; Level 1 (Beginning): Unable to apply concepts.

Clarity & Presentation (10) – Level 4 (Exemplary): Presents analysis clearly, logically, and systematically; Level 3 (Proficient): Generally clear with minor issues; Level 2 (Developing): Lacks clarity or proper structure; Level 1 (Beginning): Poor presentation and difficult to understand.